SOFTWARE TESTING AND QUALITY ASSURANCE (STQA)

MINI PROJECT REPORT

on

EduFlow – Academic Management & Productivity Portal

Web Application Testing using Selenium WebDriver, Selenium IDE, Python and pytest

Submitted by

Soniya Pawar ([Roll No])

Sakshi Jagadale ([Roll No])

Under the guidance of

[Guide Name]

Department of Computer Engineering

Academic Year: 2026 – 27

Table of Contents

Project Details....3
Theory....3
Introduction of Mini Project....3
Testing Tool: Selenium WebDriver....4
Testing Approach Used in EduFlow....4
1 TEST PLAN....4
1.1 TEST STRATEGY....4
1.2 TESTING ENVIRONMENT....5
1.3 RISK AND ISSUES....5
2 TEST OBJECTIVE....5
3 TEST CRITERIA....6
3.1 Suspension Criteria....6
3.2 Exit Criteria....6
4 RESOURCE PLANNING....6
5 TEST TOOLS....6
6 TEST CASES....6
7 TEST EXECUTION....9
8 SCREENSHOT EVIDENCE....9
8.1 Environment Setup....9
8.2 Code Implementation....10
8.3 Application Screens....13
8.4 Test Execution....16
8.5 Results....17
9 TEST RESULTS....17
10 CONCLUSION....18
11 REFERENCES / PROJECT ARTIFACTS....18

Project Details

Title: Create and test a web-based Academic Management & Productivity Portal, EduFlow, by preparing a concise test plan with the features to be tested and a bug taxonomy, narrating automated regression test scripts, identifying defects using Selenium WebDriver and Selenium IDE, and generating test reports encompassing exploratory testing.

Problem Definition: Perform web application testing for EduFlow, identify functional, validation and usability-related issues through systematic test cases and Selenium automation, and generate test reports from the actual test execution.

Prepared By:

Roll No.	Name
[Roll No]	Soniya Pawar
[Roll No]	Sakshi Jagadale

Prerequisite: Knowledge of Core Java / Python, web application fundamentals (HTML, CSS, DOM) and basic software testing concepts.

Software Requirements:

- Python 3.13 with pip
- Selenium WebDriver 4.27.1 (Selenium Manager for ChromeDriver)
- pytest 8.3.4 and pytest-html 4.1.1
- Selenium IDE browser extension
- Google Chrome browser
- PHP 8.4 and MySQL 8.0 (to run the application under test)
- Visual Studio Code
- EduFlow web application environment

Hardware Requirement: A computer/laptop (Intel Core i3 / AMD Ryzen 3 or better, 8 GB RAM, 2 GB free disk) capable of running the browser, Python, PHP, MySQL and the EduFlow application.

Theory

Introduction of Mini Project

EduFlow is a web-based Academic Management & Productivity Portal used as the application under test for this Software Testing and Quality Assurance (STQA) mini project. It has three roles – Admin, Teacher and Student. Admins manage users and classes, teachers publish assignments and grade submissions, and students submit work, manage personal tasks, read notifications, use a calendar and track their progress through charts. The testing work focuses on verifying these workflows, validating inputs and access control, identifying defects, and documenting the results with evidence from the real test run.

Testing Tool: Selenium WebDriver

Selenium is a free, open-source suite for automating web browsers. Selenium WebDriver drives the browser directly through its native automation support (ChromeDriver) using the W3C WebDriver protocol. Tests locate elements with `By` locators (ID, NAME, CSS selector, XPath), perform actions such as click and send_keys, and use explicit waits (`WebDriverWait`) to synchronise with dynamic pages. This project uses Python Selenium tests together with pytest as the test framework and pytest-html for the HTML test report. Selenium IDE, a record-and-playback browser extension, was used to record three smoke flows.

Testing Approach Used in EduFlow

- Functional testing of major EduFlow workflows for all three roles.
- Boundary value analysis and equivalence partitioning for input validation.
- Automated regression testing using Selenium WebDriver and pytest.
- Record-and-playback smoke tests using Selenium IDE.
- Browser-based validation using Google Chrome.
- Exploratory testing of edge cases (special characters, double submits, small screens).
- Generation of test execution and result reports.

1 TEST PLAN

The Test Plan defines the testing approach, resources, test environment, scope, execution strategy and expected deliverables for testing the EduFlow web application. The plan is intended to provide a structured basis for executing and documenting the functional and automated testing activities.

1.1 TEST STRATEGY

1.1.1 Scope of Testing

- Login, authentication and password recovery.
- Session handling and role-based access control.
- User and class management (Admin).
- Assignment publishing and grading (Teacher).
- Assignment submission and file-type validation (Student).
- Personal tasks, dashboard, notifications, calendar, progress analytics and profile.

1.1.2 Features to be Tested

Module	Feature	Testing Focus
Login	Authentication	Verify valid and invalid login, empty fields, case-insensitive email and lockout
Access Control	Session & roles	Verify logout, direct URL access without login and 403 for wrong role
Password Recovery	Forgot / reset	Verify email format validation and invalid reset links
Users & Classes	Admin workflow	Verify user creation, duplicate email, password boundaries, search, deactivation
Assignments	Teacher workflow	Verify create/publish, required fields, due-date rule and grading limits
Submissions	Student workflow	Verify PDF upload and rejection of disallowed file types
Tasks & Dashboard	Productivity	Verify quick-add, completion, title length, card navigation, notifications

Module	Feature	Testing Focus
Calendar & Progress	Views & charts	Verify month/agenda views, chart rendering and period filter

1.1.3 Test Types

- Functional Testing
- Integration Testing
- System Testing
- Regression Testing
- UI / Navigation Validation
- Exploratory Testing

1.2 TESTING ENVIRONMENT

Item	Environment / Tool
Operating Environment	Windows 11 (64-bit)
Application URL	http://127.0.0.1:8000 (PHP built-in server)
Application Stack	PHP 8.4.5, MySQL 8.0, Bootstrap 5.3, Chart.js 4.4
Browser	Google Chrome 154 (headless and normal mode)
Programming Language	Python 3.13
Automation Tool	Selenium WebDriver 4.27.1, Selenium IDE
Test Framework / Report	pytest 8.3.4, pytest-html 4.1.1
IDE	Visual Studio Code
Test Data	Demo seed database; password Demo@1234 for all demo accounts

1.3 RISK AND ISSUES

Risk	Impact / Issue	Mitigation
Tests change data (new users, submissions), so a second run behaves differently	Flaky or failing re-runs	Reset the database to the demo seed before every full run (`php bin/migrate.php --fresh` + `php bin/seed.php`)
Login lockout (5 failed attempts in 15 min) blocks accounts used by other tests	Unrelated tests fail	Lockout test uses a dedicated account not used elsewhere; database reset clears counters
Timing issues with JavaScript-loaded content	False failures	Explicit waits (`WebDriverWait`) on visible elements; no fixed sleeps
ChromeDriver / Chrome version mismatch	Suite cannot start	Selenium Manager downloads the matching driver automatically
Team members unfamiliar with Selenium	Slow script writing	Shared helper module (login, waits, screenshots) and Selenium IDE recordings as examples
Limited time before submission	Incomplete testing	Prioritise login, access control and the assignment workflow first

2 TEST OBJECTIVE

The main objective of the project is to verify the functionality of the EduFlow web application and demonstrate automated regression testing using Selenium WebDriver. The project also aims to document test cases, record results, find and report defects with clear reproduction steps, and check that validation and access control protect the users' data.

3 TEST CRITERIA

3.1 Suspension Criteria

- 40% or more of the test cases fail.
- The EduFlow application or database is unavailable for a significant part of the planned execution.
- A blocking defect (no role can log in) prevents execution of the majority of the planned test cases.

3.2 Exit Criteria

- Run rate is 100%: all documented test cases have been executed.
- Pass rate is at least 80%.
- No Critical defect is left open.
- Test results, screenshots and execution evidence have been collected.
- The final test report is generated and reviewed.

4 RESOURCE PLANNING

Resource	Requirement	Purpose
Hardware	Computer/Laptop (8 GB RAM)	Development, application execution and automated testing
Software	Python + Selenium + Chrome	Automation environment
Server	PHP 8.4 + MySQL 8.0	Run the EduFlow application under test
Human – Test Lead	Soniya Pawar	Test plan, bug taxonomy, automation framework, final report
Human – Test Engineer	Sakshi Jagadale	Test case design, Selenium scripts, Selenium IDE recordings, exploratory testing, bug reports

5 TEST TOOLS

Tool	Purpose
Selenium WebDriver	Automates browser interaction and executes the regression test scripts
Python	Programming language used for the Selenium automation code
pytest	Test framework: fixtures, assertions, test discovery and execution
pytest-html	Generates the HTML test execution report
Selenium IDE	Records and replays browser flows (EduFlow.side)
Google Chrome	Browser used for test execution

6 TEST CASES

The project contains 36 documented test cases (TC01–TC36), covering Authentication, User & Class Management, Assignments & Grading, Submissions, Personal Tasks, Dashboard & Analytics and Profile. All 36 test cases were automated with Selenium WebDriver and executed in one regression run; the Actual Result and Status columns are taken from that run.

TC ID	Test Objective	Test Steps	Expected Result	Actual Result	Status
TC01	Authentication: Valid login as Admin	Open /login; email `admin@eduflow.test`, password `Demo@1234`; click	Redirected to /admin; page title "Overview"	Redirected to /admin; heading 'Overview'	**Pass**

TC ID	Test Objective	Test Steps	Expected Result	Actual Result	Status
		Log in			
TC02	Authentication: Valid login as Teacher	Email `meera.teacher@eduflow.test`, valid password	Redirected to /teacher; Dashboard shown	Redirected to /teacher; heading 'Dashboard'	**Pass**
TC03	Authentication: Valid login as Student	Email `priya.student@eduflow.test`, valid password	Redirected to /student; Dashboard with summary cards	Redirected to /student; 7 summary cards shown	**Pass**
TC04	Authentication: Login with wrong password	Valid email, password `WrongPass1`	Stays on login page; "Incorrect email or password." shown	Stayed on /login; message: 'Incorrect email or password.'	**Pass**
TC05	Authentication: Login with unregistered email	Email `nobody@eduflow.test`, any password	Same generic error; no hint that the email does not exist	Stayed on /login; same generic message: 'Incorrect email or password.'	**Pass**
TC06	Authentication: Login with empty fields	Leave email and password empty; click Log in	Not logged in; error message shown	Not logged in; message: 'Incorrect email or password.'	**Pass**
TC07	Authentication: Email is case-insensitive	Email `PRIYA.STUDENT@EDUFLOW.TEST`, valid password	Logged in, redirected to /student	Upper-case email accepted; redirected to /student	**Pass**
TC08	Authentication: Account lockout after 5 failed attempts (boundary)	Enter wrong password 5 times for `isha.student@eduflow.test`, then the correct password	6th attempt refused with "Too many login attempts"	5 failures accepted as errors; 6th (correct password) refused: 'Too many login attempts. Please wait 15 minutes and try again.'	**Pass**
TC09	Authentication: Logout ends the session	Log in as student; open profile menu; Log out; then open /student	Redirected to /login after logout; /student redirects to /login	After logout at /login; reopening /student redirected to /login?next=%2Fstudent	**Pass**
TC10	Authentication: Direct URL access without login	Without logging in, open /admin/users	Redirected to /login?next=…	Redirected to /login?next=%2Fadmin%2Fusers	**Pass**
TC11	Authentication: Role restriction	Log in as student; open /admin/users	"403 · Access denied" page	Page shows '403 · Access denied'	**Pass**
TC12	Authentication: Forgot password with valid email	/forgot-password; email `priya.student@eduflow.test`; Send	Redirected to /login with "If an account exists…" notice	Redirected to /login with notice: 'If an account exists for that email, a reset link has been sent. It expires in 1 hour.'	**Pass**
TC13	Authentication: Forgot password with invalid email format	Email `not-an-email`; Send	"Enter a valid email address." shown	Message: 'Enter a valid email address.'	**Pass**
TC14	Authentication: Reset password with invalid link	Open /reset-password?token=abc123	"This reset link is invalid or has expired." and link to request a new one	Message: 'This reset link is invalid or has expired.'; link offered: 'Request a new link'	**Pass**
TC15	User & Class Management: Admin creates a student (happy path)	Admin → Students → Add student; name `Test Student`, unique email, password `Secure123`, admission no `STQA001`; Save	Toast "Account created."; student appears in search results	Toast 'Account created.'; student found by admission number	**Pass**
TC16	User & Class Management: Duplicate email rejected	Add student with email `priya.student@eduflow.test`	Field error "An account with this email already exists."	Field error: 'An account with this email already exists.'	**Pass**
TC17	User & Class Management: Password below minimum length (boundary 7)	Add student with password `Abcde12` (7 chars)	Field error "Password must be at least 8 characters."	Field error: 'Password must be at least 8 characters.'	**Pass**
TC18	User & Class Management: Password at minimum length	Add student with password `Abcdef12` (8 chars), unique email	Account created	8-character password accepted; toast 'Account created.'	**Pass**

TC ID	Test Objective	Test Steps	Expected Result	Actual Result	Status
	(boundary 8)				
TC19	User & Class Management: Admin creates a class	Admin → Classes → Add class; name `SY BSc CS`, division `C`; Save	Toast "Class created."; class listed	Toast 'Class created.'; 'SY BSc CS C' listed	**Pass**
TC20	User & Class Management: Admin searches users	Admin → Users; type `Kulkarni` in search	Only "Meera Kulkarni" listed	Results: ['Meera Kulkarni']	**Pass**
TC21	User & Class Management: Admin deactivates a user, who then cannot log in	Admin → Students; deactivate `Dev Patil`; confirm; then log in as `dev.student@eduflow.test`	Status shows "Deactivated"; login shows "This account has been deactivated"	Status 'Deactivated' shown; login message: 'This account has been deactivated. Contact your administrator.'	**Pass**
TC22	Assignments & Grading: Teacher creates and publishes an assignment	Teacher → Assignments → New; title, instructions, class & subject, due date; Save & publish	Assignment page opens with "Published" badge	Assignment page opened: 'Recursion practice set' with Published badge	**Pass**
TC23	Assignments & Grading: Assignment required fields	New assignment; leave everything empty; Save as draft	Errors under Title, Instructions and Class & subject; not saved	Title: 'This field is required.', Instructions: 'This field is required.', Class & subject: 'no message', top banner: 'This field is required. This field is required.'	**Fail**
TC24	Assignments & Grading: Publish without due date rejected	New assignment with title, instructions, class; no due date; Save & publish	Error "Set a due date before publishing…"	Field error: 'Set a due date before publishing an assignment that needs a submission.'	**Pass**
TC25	Assignments & Grading: Marks above maximum rejected (boundary)	Teacher opens Sara Khan's "ER diagram" submission (max 20); marks `21`; Save grade	Error "Marks cannot exceed 20."	Field error: 'Marks cannot exceed 20.'	**Pass**
TC26	Assignments & Grading: Teacher grades a submission	Same submission; marks `18`, feedback; Save grade	Status changes to "Graded"; toast "Grade saved."	Toast 'Grade saved.'; status badge 'Graded', marks 18 / 20	**Pass**
TC27	Assignment Submission: Student submits a valid PDF	Student Priya → My Assignments → "Normalization case study"; upload `answer.pdf`; Submit	Toast "Submitted…"; status "Submitted", file listed	Toast 'Submitted. Your teacher can see it now.'; card shows 'Your submission Submitted' and answer.pdf	**Pass**
TC28	Assignment Submission: Disallowed file type rejected	Student Sara → same assignment; upload `notes.txt`; Submit	Error "…is not an allowed file type…"; nothing saved	Field error: '"notes.txt" is not an allowed file type. Allowed: pdf, docx.'; status still 'Not submitted'	**Pass**
TC29	Personal Tasks: Quick-add a personal task	Student → Personal Tasks; type `Revise OS notes tomorrow 6pm high priority`; Add	Toast `Added "Revise OS notes"`; task listed with High priority	Preview 'Revise OS notes'; toast 'Added "Revise OS notes".'; listed with High priority	**Pass**
TC30	Personal Tasks: Complete a personal task	Tick the checkbox next to a pending personal task	Toast "marked complete"; task status Completed	'Revise OS notes' ticked; toast 'Nice — marked complete.'; now in Completed list	**Pass**
TC31	Personal Tasks: Task title maximum length (boundary 200)	New personal task; type 250 characters in Title; Save	Field accepts only 200 characters; task saved	Typed 250 characters, field kept 200; toast 'Task added.'	**Pass**
TC32	Dashboard & Analytics: Dashboard card navigation	Student dashboard; click the "Overdue" card	Personal Tasks page opens with Status filter = Overdue	Overdue card (1) opened /student/tasks?status=overdue with filter 'Overdue'	**Pass**
TC33	Dashboard & Analytics: Mark all notifications as read	Student → Notifications → Mark all as read	Toast "Marked N as read."; unread badge disappears	Badge was 5; toast 'Marked 5 as read.'; badge hidden, no unread items	**Pass**
TC34	Dashboard & Analytics: Calendar views	Student → Calendar; switch to Agenda	Month grid shown first; agenda list shown after switching	Month grid with 42 days and 74 events; agenda view shows 24 days	**Pass**
TC35	Dashboard & Analytics: Progress	Student → My Progress; change Period to "This	Charts render without errors; date range text	5 charts rendered, no errors; range 'Oct 1, 2026	**Pass**

TC ID	Test Objective	Test Steps	Expected Result	Actual Result	Status
	charts and period filter	week"	changes	– Oct 31, 2026' changed to 'Oct 12, 2026 – Oct 18, 2026'	
TC36	Profile: Profile update and wrong current password	Profile; change name, Save; then change password with wrong current password	Toast "Profile saved."; error "Current password is incorrect."	Toast 'Profile saved.'; name persisted after refresh; password error 'Current password is incorrect.'	**Pass**

7 TEST EXECUTION

The automated suite is executed with pytest. The single command `pytest` starts the EduFlow server, resets the database to the demo seed, opens Chrome through Selenium WebDriver and performs every test case, saving a screenshot on failure and writing the HTML report to `reports/test_report.html`. The Selenium IDE flows are replayed with `python selenium_ide/replay_side.py`.

1. Install the requirements: `pip install -r requirements.txt`.
2. Verify the Python, PHP and MySQL installation.
3. Open the automation project in Visual Studio Code.
4. Execute the test command `pytest`.
5. Observe Selenium-driven browser execution.
6. Verify console output and final test statistics.
7. Open the generated pytest-html report.

8 SCREENSHOT EVIDENCE

This section presents the screenshot evidence collected during the project, grouped into environment setup, code implementation, application screens, test execution and results.

8.1 Environment Setup

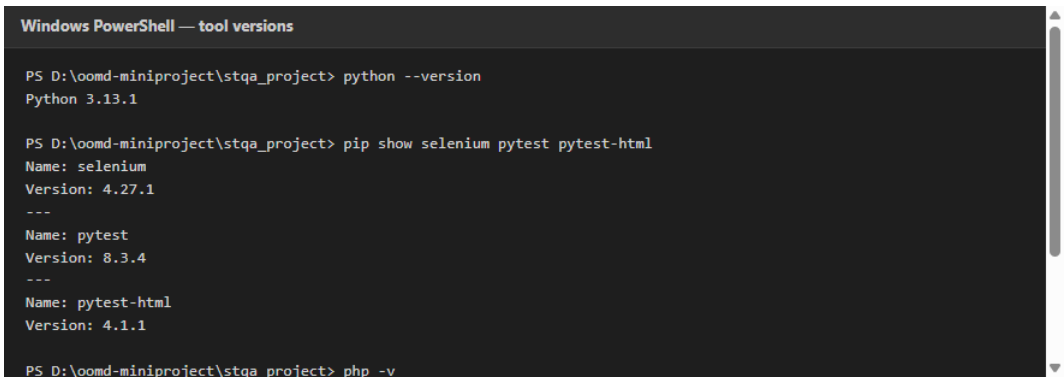


Figure 1: Python, Selenium, pytest, PHP and MySQL version output

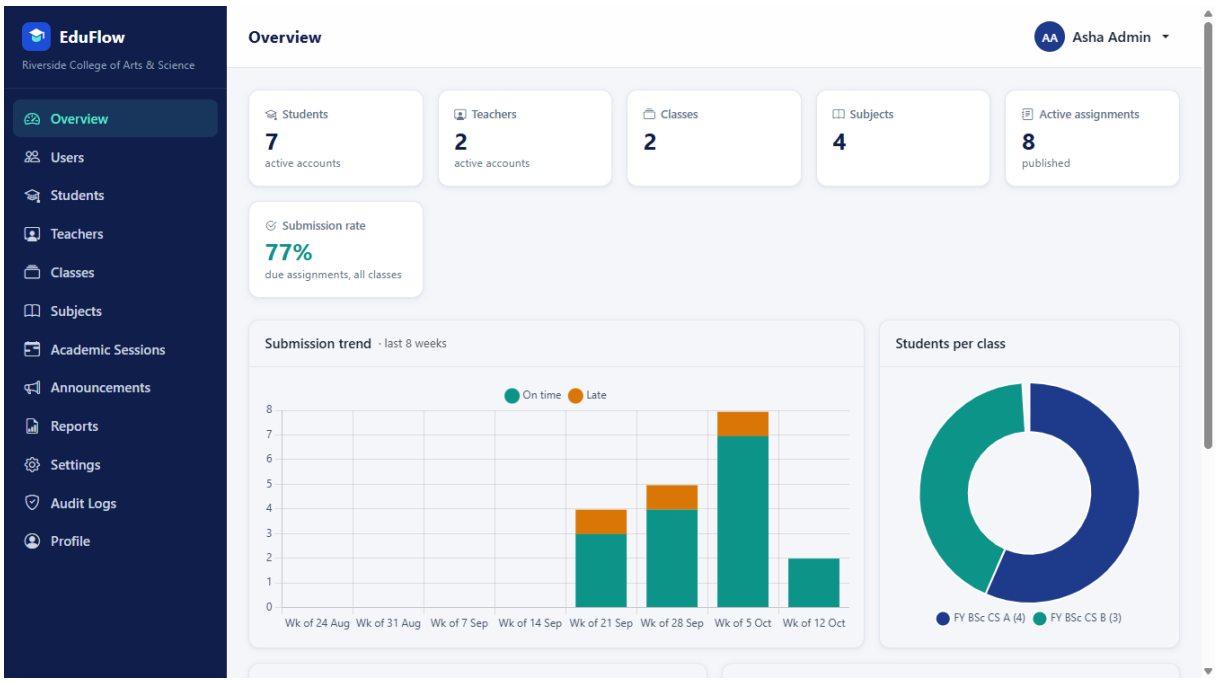


Figure 2: EduFlow application ready (Admin overview page)

8.2 Code Implementation

```
class App:
    """Thin wrapper around the driver with the base URL and common actions."""

    def __init__(self, driver: WebDriver, base_url: str):
        self.driver = driver
        self.base = base_url.rstrip("/")
        self.wait = WebDriverWait(driver, TIMEOUT)

    # ------------------------------------------------------------- navigation
    def open(self, path: str) -> None:
        self.driver.get(self.base + path)

    def path(self) -> str:
        return self.driver.current_url.replace(self.base, "") or "/"

    def wait_url_contains(self, fragment: str) -> None:
        self.wait.until(EC.url_contains(fragment))

    # ------------------------------------------------------------- finding
    def visible(self, by: str, value: str) -> WebElement:
        return self.wait.until(EC.visibility_of_element_located((by, value)))

    def clickable(self, by: str, value: str) -> WebElement:
        return self.wait.until(EC.element_to_be_clickable((by, value)))

    def css(self, selector: str) -> WebElement:
        return self.visible(By.CSS_SELECTOR, selector)

    def click(self, selector: str) -> None:
        self.clickable(By.CSS_SELECTOR, selector).click()

    def scroll_click(self, element: WebElement) -> None:
        """Scroll an element to the middle of the viewport, wait until clickable, then click."""
        self.driver.execute_script("arguments[0].scrollIntoView({block: 'center'});", element)
        self.wait.until(EC.element_to_be_clickable(element)).click()

    def text_present(self, selector: str, text: str) -> bool:
```

```
        return self.wait.until(EC.text_to_be_present_in_element((By.CSS_SELECTOR, selector),
text))

    def page_text(self) -> str:
        return self.driver.find_element(By.TAG_NAME, "body").text

    # ------------------------------------------------------------ auth
    def login(self, email: str, password: str = PASSWORD) -> None:
        self.open("/login")
        self.css("#email").clear()
        self.css("#email").send_keys(email)
        self.css("#password").clear()
        self.css("#password").send_keys(password)
        self.click("form button[type=submit]")

    def login_as(self, email: str, landing: str) -> None:
        self.login(email)
        self.wait_url_contains(landing)
        self.css("h1.page-title")

    def logout(self) -> None:
        self.click(".btn-profile")
        self.clickable(By.XPATH, "//button[normalize-space()='Log out']").click()
        self.wait_url_contains("/login")

    def login_error(self) -> str:
```

Figure 3: helpers.py – reusable App class with explicit waits, login and screenshot helpers

```
@pytest.fixture(scope="session", autouse=True)
def fresh_database(request, base_url):
    """Reset to the demo seed so every full run starts from the same data."""
    if request.config.getoption("--no-reset"):
        return
    for cmd in (["php", "bin/migrate.php", "--fresh"], ["php", "bin/seed.php"]):
        result = subprocess.run(cmd, cwd=APP_ROOT, capture_output=True, text=True)
        if result.returncode != 0:
            pytest.exit(f"Database reset failed: {'
'.join(cmd)}\n{result.stdout}{result.stderr}", returncode=3)



@pytest.fixture
def driver(request):
    opts = webdriver.ChromeOptions()
    if not request.config.getoption("--headed"):
        opts.add_argument("--headless=new")
    opts.add_argument("--window-size=1366,900")
    opts.add_argument("--disable-search-engine-choice-screen")
    drv = webdriver.Chrome(options=opts)
    yield drv
    drv.quit()



@pytest.fixture
def app(driver, base_url):
    return App(driver, base_url)



@pytest.fixture
def actual(request):
```

Figure 4: conftest.py – fixtures for server start-up, database reset and Chrome driver

```
def test_tc04_wrong_password(app, actual):
    app.login(STUDENT, "WrongPass1")
    error = app.login_error()
    assert app.path() == "/login"
    assert "Incorrect email or password" in error
    actual(f"Stayed on /login; message: '{error}'")
```

```
def test_tc05_unregistered_email(app, actual):
    app.login("nobody@eduflow.test", "Whatever123")
    error = app.login_error()
    assert app.path() == "/login"
    assert "Incorrect email or password" in error
    actual(f"Stayed on /login; same generic message: '{error}'")


def test_tc06_empty_fields(app, actual):
    app.login("", "")
    error = app.login_error()
    assert app.path() == "/login"
    assert error
    actual(f"Not logged in; message: '{error}'")


def test_tc07_email_case_insensitive(app, actual):
    app.login(STUDENT.upper())
    app.wait_url_contains("/student")
    assert app.path() == "/student"
    actual("Upper-case email accepted; redirected to /student")


def test_tc08_lockout_after_five_failures(app, actual):
    email = "isha.student@eduflow.test"  # used only by this test
    for _ in range(5):
        app.login(email, "WrongPass1")
        assert "Incorrect email or password" in app.login_error()
    app.login(email, PASSWORD)
    error = app.login_error()
    app.shot("TC08_lockout")
    assert "Too many login attempts" in error
    actual(f"5 failures accepted as errors; 6th (correct password) refused: '{error}'")


def test_tc09_logout_ends_session(app, actual):
    app.login_as(STUDENT, "/student")
    app.logout()
    after_logout = app.path()
    app.open("/student")
    app.wait_url_contains("/login")
    assert after_logout.startswith("/login")
    assert app.path().startswith("/login")
    actual(f"After logout at {after_logout}; reopening /student redirected to {app.path()}")


def test_tc10_direct_url_without_login(app, actual):
    app.open("/admin/users")
    app.wait_url_contains("/login")
    assert app.path() == "/login?next=%2Fadmin%2Fusers"
    actual(f"Redirected to {app.path()}")


def test_tc11_student_cannot_open_admin_page(app, actual):
```

Figure 5: Selenium test code – login, lockout and access-control tests (TC04–TC11)

```
def test_tc22_create_and_publish_assignment(app, actual):
    new_assignment_form(app)
    due = (datetime.now() + timedelta(days=5)).strftime("%Y-%m-%dT17:00")
    app.modal_type("title", "Recursion practice set")
    app.modal_type("description", "Solve the five recursion problems and upload a PDF.")
    app.modal_select("teaching", text=CLASS_SUBJECT)
    app.modal_set_value("due_at", due)
    app.modal_type("max_marks", "10")
```

```
    save_and_publish(app)
    app.wait_url_contains("/teacher/assignments/")
    header = app.css("#assignment-detail .card-header").text
    app.shot("TC22_assignment_published")
    assert "Recursion practice set" in header and "Published" in header
    actual(f"Assignment page opened: '{header.splitlines()[0]}' with Published badge")


def test_tc23_assignment_required_fields(app, actual):
    new_assignment_form(app)
    app.modal_submit()  # "Save as draft" with nothing filled in
    app.modal_error("title")  # wait for the validation response
    errors = {
        f: app.driver.find_element(By.CSS_SELECTOR, f".modal.show [data-error-for='{f}']").text
        for f in ("title", "description", "teaching")
    }
    banner = app.driver.find_element(By.CSS_SELECTOR, ".modal.show [data-form-error]")
    banner_text = banner.text if banner.is_displayed() else ""
    app.shot("TC23_required_fields")
    observed = (f"Title: '{errors['title']}', Instructions: '{errors['description']}', "
                f"Class & subject: '{errors['teaching'] or 'no message'}', top banner:
'{banner_text or 'none'}'")
    actual(observed)
    assert all(e == "This field is required." for e in errors.values()), observed


def test_tc24_publish_without_due_date(app, actual):
    new_assignment_form(app)
    app.modal_type("title", "No deadline task")
    app.modal_type("description", "Anything.")
    app.modal_select("teaching", text=CLASS_SUBJECT)
    save_and_publish(app)
    error = app.modal_error("due_at")
```

Figure 6: Selenium test code – create and publish assignment (TC22)

8.3 Application Screens

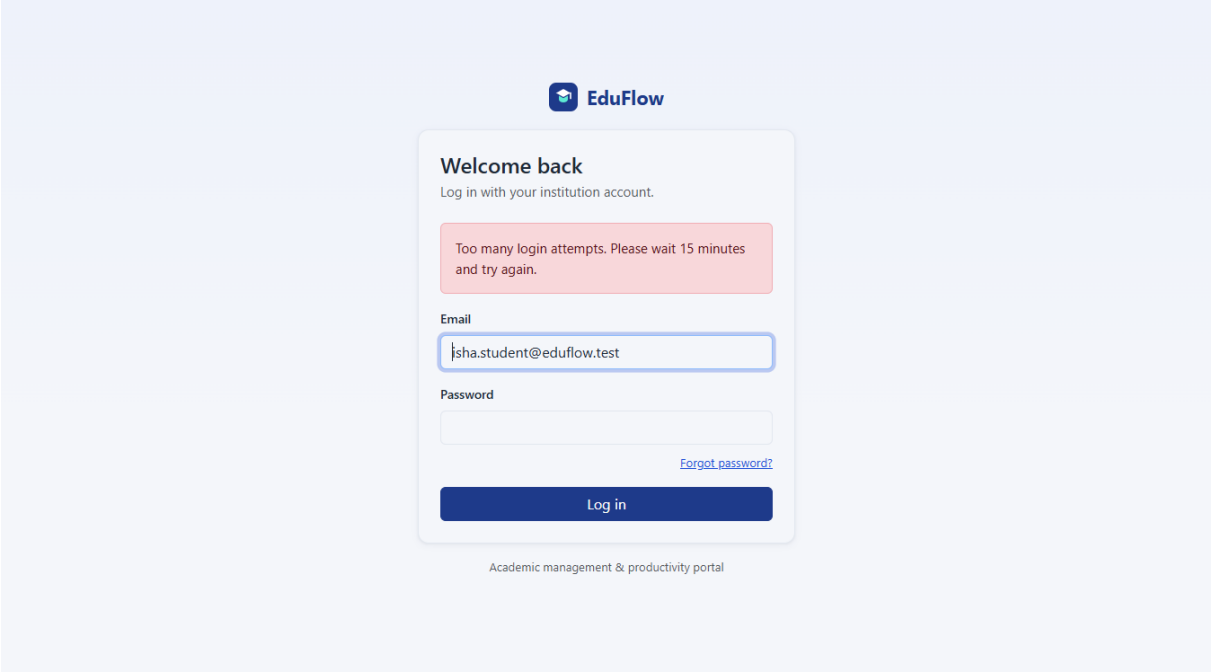


Figure 7: EduFlow login page – lockout after 5 failed attempts (TC08)

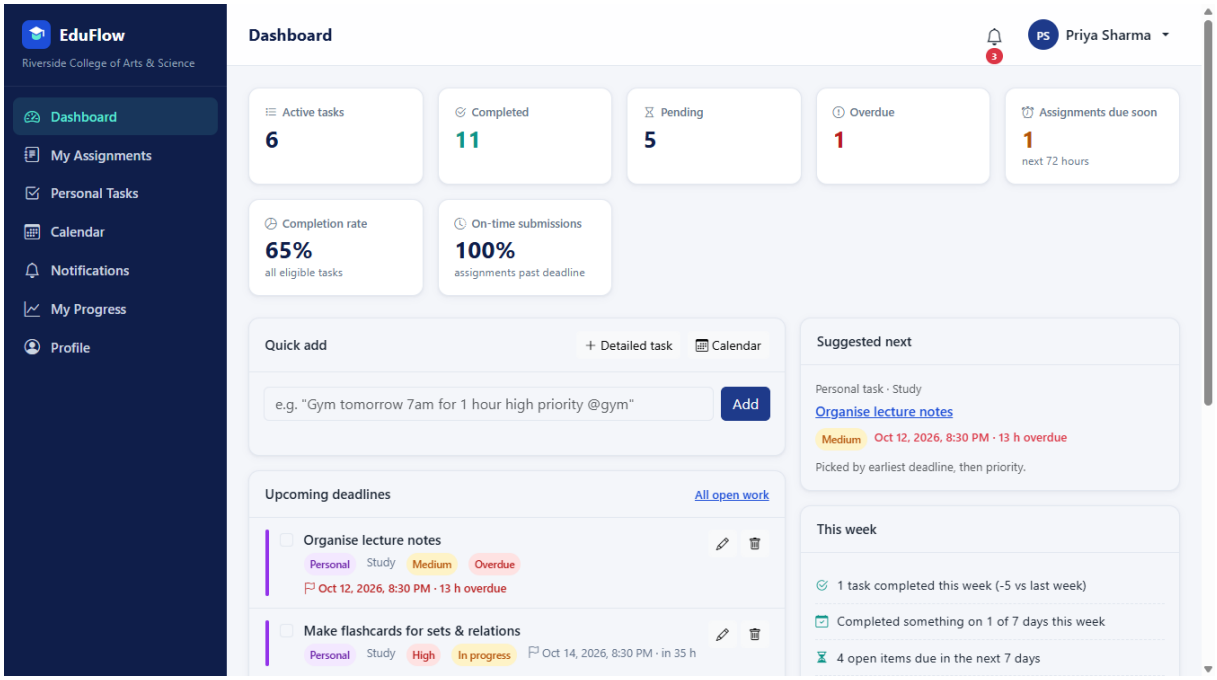


Figure 8: EduFlow student dashboard (TC03)

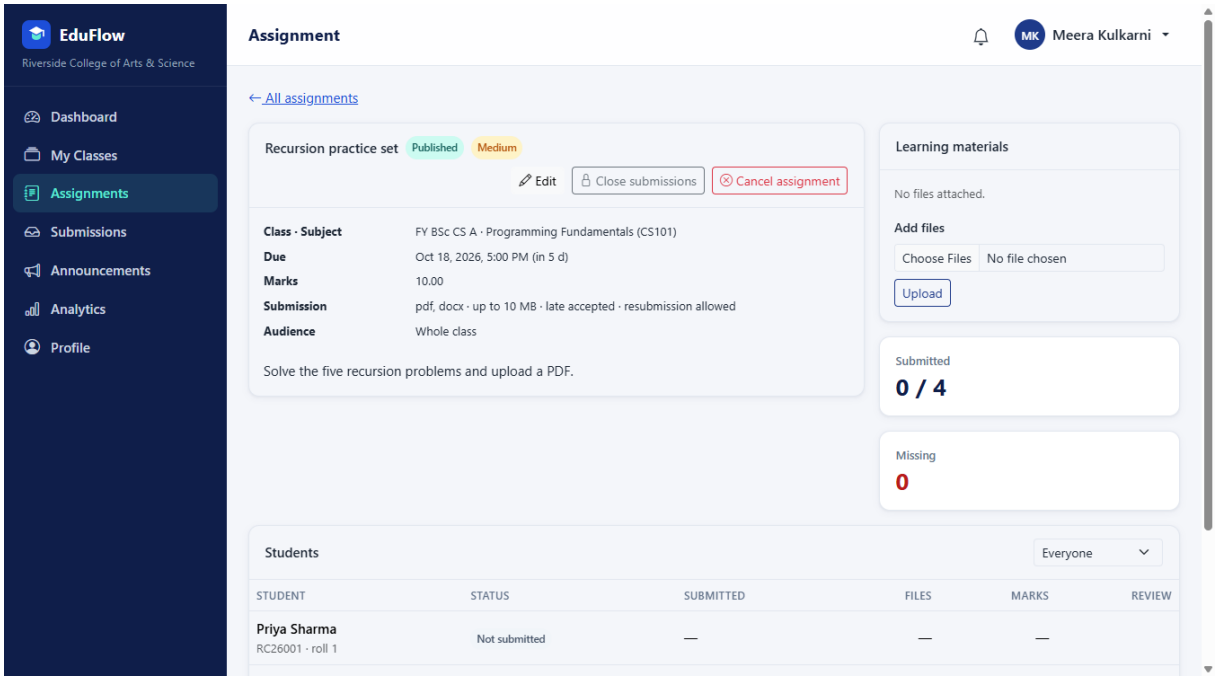


Figure 9: Teacher's assignment published (TC22)

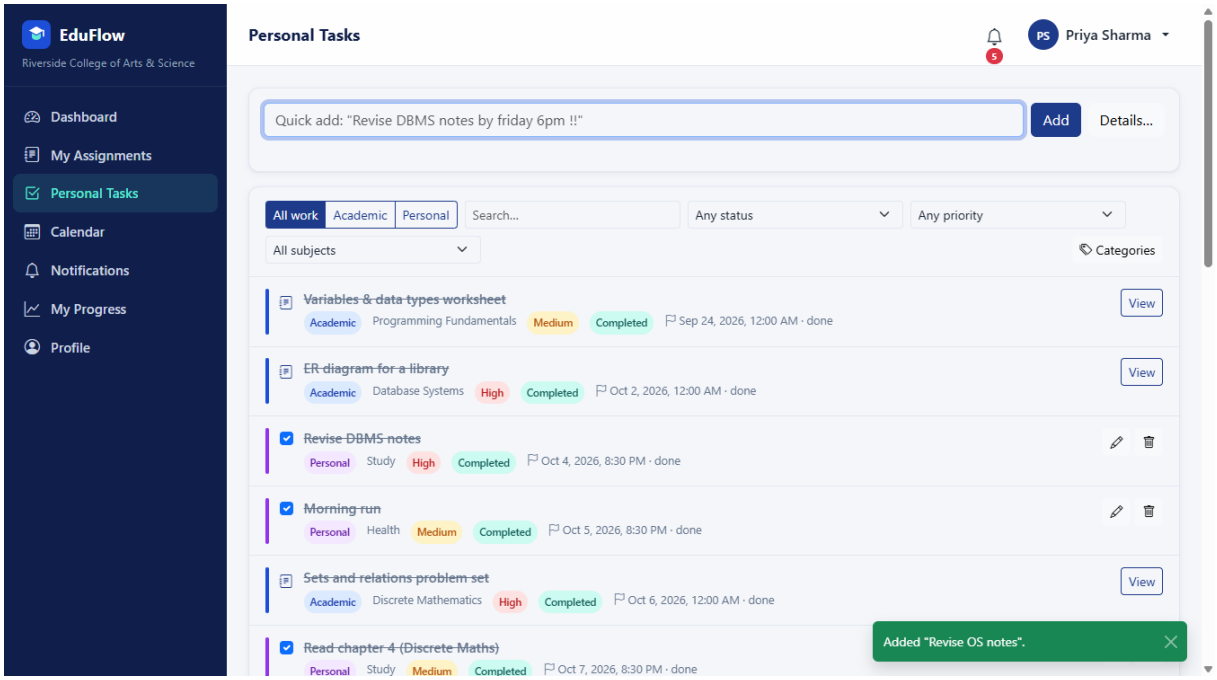


Figure 10: Quick-add personal task (TC29)

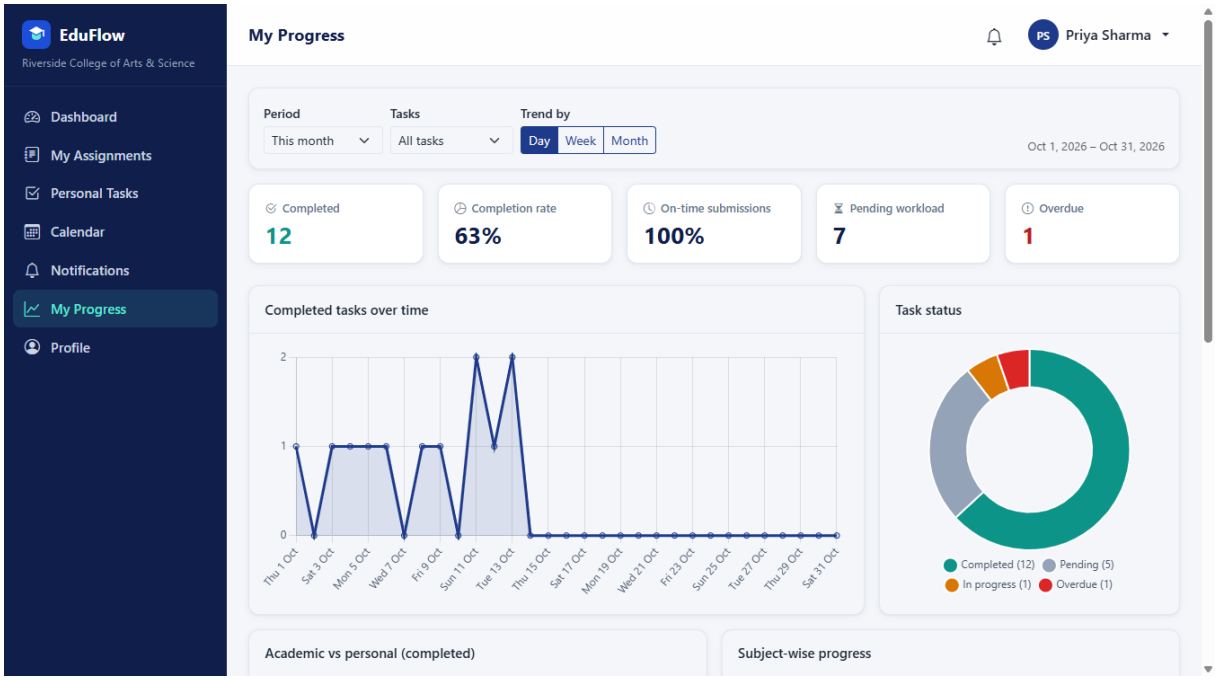


Figure 11: Student progress analytics dashboard (TC35)

8.4 Test Execution

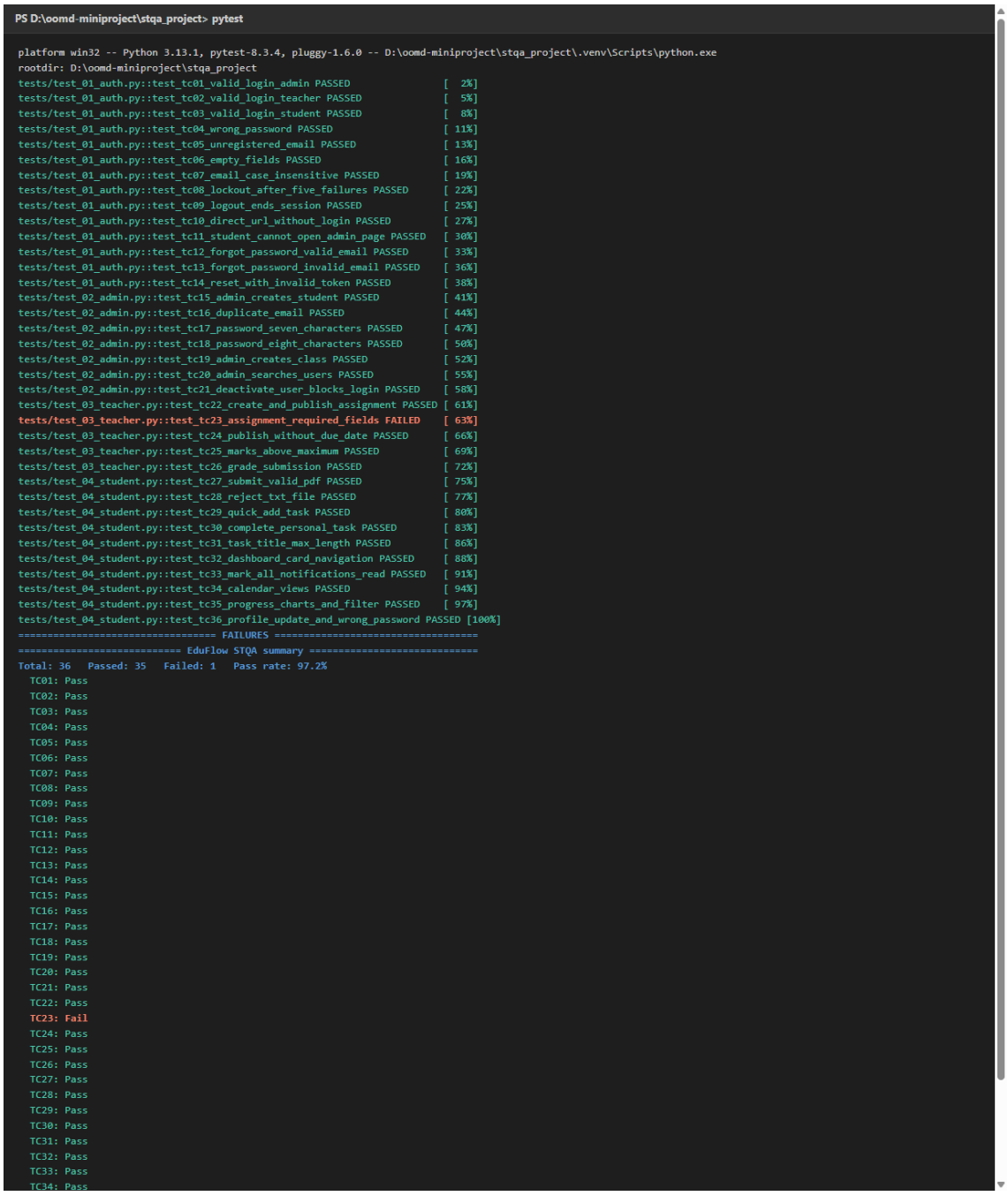


Figure 12: Console showing the full regression run and final results

Figure 13: Selenium IDE flows replayed – 3 of 3 passed

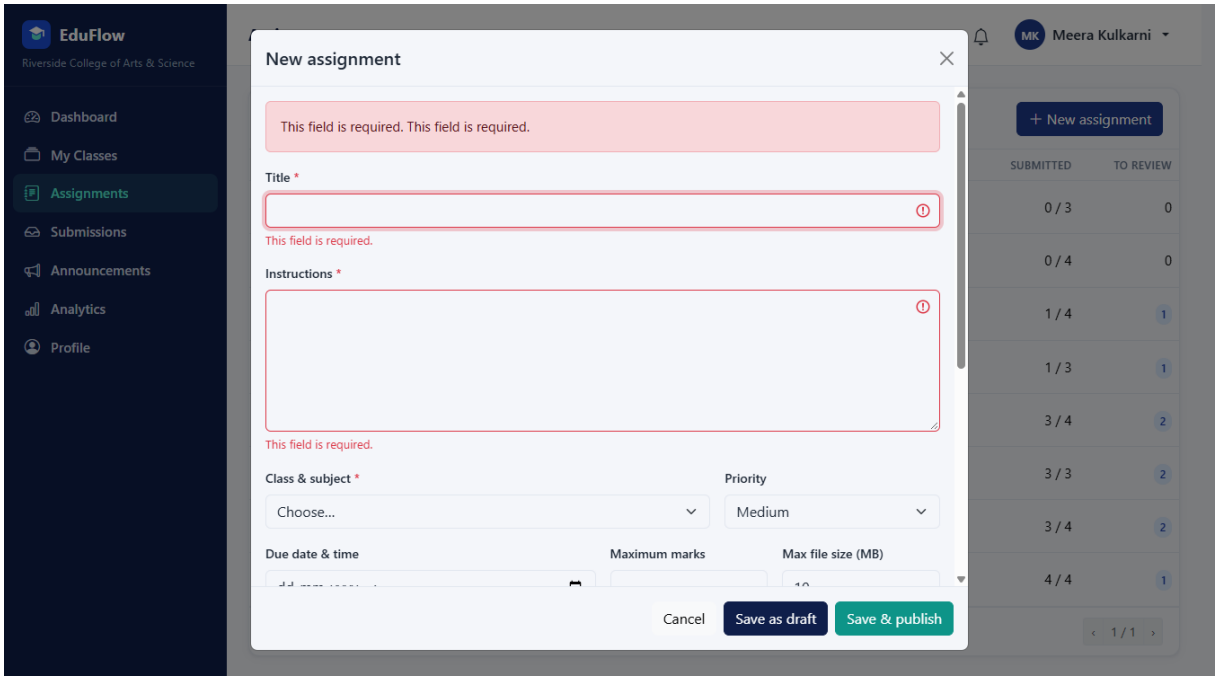


Figure 14: Failed test TC23 – no required-field message under Class & subject (BUG-01)

8.5 Results

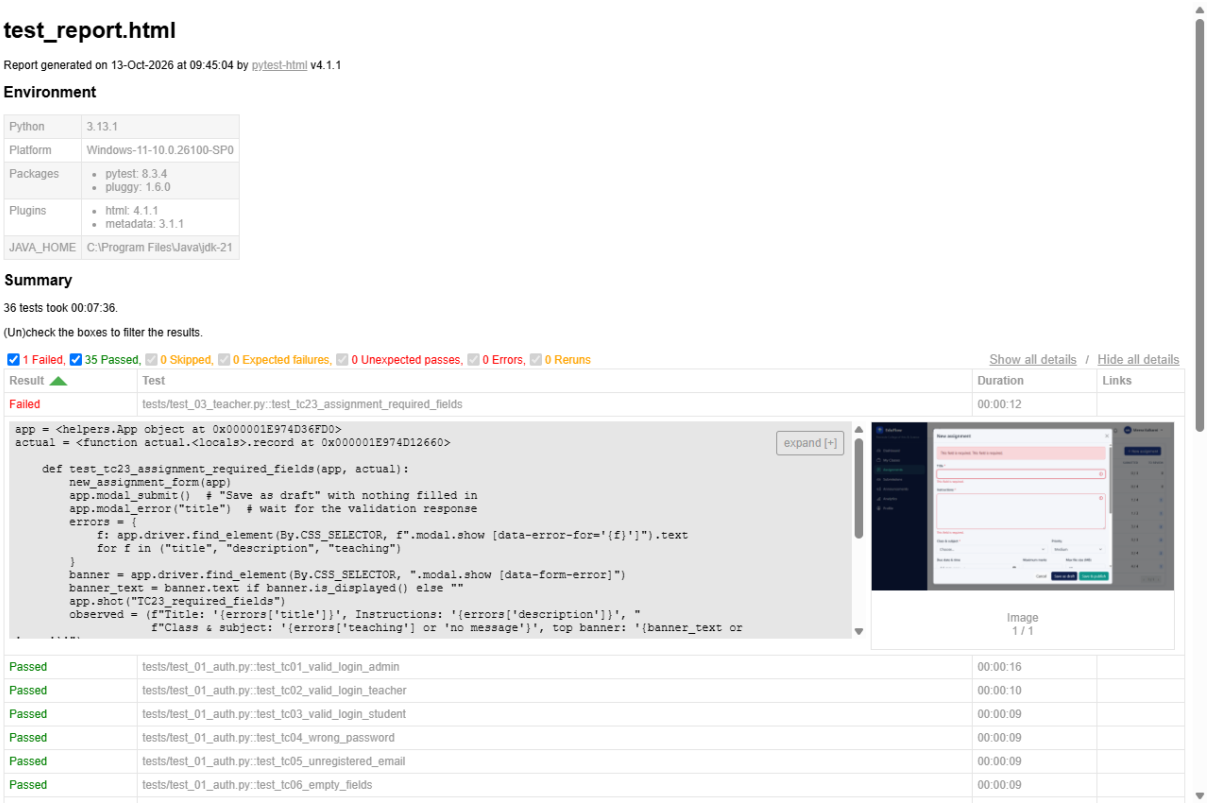


Figure 15: pytest-html report – full suite results

9 TEST RESULTS

Across all 36 documented test cases (TC01–TC36), 35 passed and 1 failed, as recorded in the Test Cases table in Section 6. All 36 were automated using Selenium WebDriver and pytest and executed in a single regression run from a freshly seeded database.

Metric	Observed Result
Total tests	36
Passed	35
Failed	1 (TC23)
Run rate	100%
Pass rate	97.2%
Execution time	3.2 minutes
Selenium IDE flows	3 of 3 passed

Verified overall result: 35 of 36 test cases passed and 1 failed (97.2% pass rate), which meets the exit criteria (100% run rate, at least 80% pass rate, no open Critical defect). Two Minor defects were reported:

Bug ID	Title	Severity	Priority	Status
BUG-01	"Class & subject" required error not shown under the field (found by TC23)	Minor	P3	Open
BUG-02	Double-clicking "Add" in quick-add creates duplicate tasks (exploratory)	Minor	P3	Open

10 CONCLUSION

This STQA mini project applied systematic software testing practices to the EduFlow web application. The report documents the testing scope, environment, resources, test cases, automation approach and results. All 36 test cases were automated with Selenium WebDriver and pytest using explicit waits, stable locators and automatic failure screenshots; 35 passed and one exposed a real validation defect. Exploratory testing found a second defect and confirmed that access control, input escaping, length limits and responsive layouts work correctly.

The automated suite runs unattended in about eight minutes from a clean database, so it can be repeated after every change as a regression check. Selenium WebDriver with pytest proved suitable for testing a modern JavaScript-driven web application, and the suite can be extended with Selenium Grid for cross-browser testing in the future.

11 REFERENCES / PROJECT ARTIFACTS

- EduFlow project application
- EduFlow Selenium test automation code (`tests/`)
- pytest configuration (`pytest.ini`, `requirements.txt`)
- Selenium IDE project (`selenium_ide/EduFlow.side`)
- EduFlow test plan, test cases and bug taxonomy (`test_plan.md`, `test_cases.md`, `bug_taxonomy.md`)
- Bug report and exploratory testing notes (`bug_report.md`, `exploratory_testing.md`)
- EduFlow test-execution report (`reports/test_report.html`)
- STQA mini project report reference format supplied for this submission